Node Embeddings for Java

This notebook demonstrates different methods for node embeddings and how to further reduce their dimensionality to be able to visualize them in a 2D plot.

Node embeddings are essentially an array of floating point numbers (length = embedding dimension) that can be used as "features" in machine learning. These numbers approximate the relationship and similarity information of each node and can also be seen as a way to encode the topology of the graph.

Considerations

Due to dimensionality reduction some information gets lost, especially when visualizing node embeddings in two dimensions. Nevertheless, it helps to get an intuition on what node embeddings are and how much of the similarity and neighborhood information is retained. The latter can be observed by how well nodes of the same color and therefore same community are placed together and how much bigger nodes with a high centrality score influence them.

If the visualization doesn't show a somehow clear separation between the communities (colors) here are some ideas for tuning:

- Clean the data, e.g. filter out very few nodes with extremely high degree that aren't actually that important
- Try directed vs. undirected projections
- Tune the embedding algorithm, e.g. use a higher dimensionality
- Tune UMAP that is used to reduce the node embeddings dimension to two dimensions for visualization.

It could also be the case that the node embeddings are good enough and well suited the way they are despite their visualization for the down stream task like node classification or link prediction. In that case it makes sense to see how the whole pipeline performs before tuning the node embeddings in detail.

Note about data dependencies

PageRank centrality and Leiden community are also fetched from the Graph and need to be calculated first. This makes it easier to see if the embeddings approximate the structural information of the graph in the plot. If these properties are missing you will only see black dots all of the same size.

References

- jqassistant
- Neo4j Python Driver
- Tutorial: Applied Graph Embeddings
- Visualizing the embeddings in 2D
- UMAP
- AttributeError: 'list' object has no attribute 'shape'
- Fast Random Projection (neo4j)
- HashGNN (neo4j)
- node2vec (neo4j) computes a vector representation of a node based on second order random walks in the graph.
- Complete guide to understanding Node2Vec algorithm

```
The numpy version is: 1.26.4
The pandas version is: 2.2.3
The UMAP version is: 0.5.9.post2
The matplotlib version is: 3.10.8
The sklearn version is: 1.6.1
```

1. Java Packages

1.1 Create Dependency Graph Projection for Java Packages

The projection and related common parameters are shared across all embedding algorithms below.

	nodeCount	relationshipCount	density	sizeInBytes	degreeDistribution.min	degreeDistribution.mean	degreeDistribution.max	degreeDistri
0	120	1528	0.107003	2663439	1	12.733333	79	

1.2 Generate Node Embeddings using Fast Random Projection (Fast RP) for Java Packages

Fast Random Projection is used to reduce the dimensionality of the node feature space while preserving most of the distance information. Nodes with similar neighborhood result in node embedding with similar vectors.

👉**Hint:** To skip existing node embeddings and always calculate them based on the parameters below edit `Node_Embeddings_0a_Query_Calculated` so that it won't return any results.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The p
rovided relationship type is not in the database.} {description: One of the relations
hip types in your query is not available in the database, make sure you didn't misspe
ll it or that the label is available when you run this statement in your application
(the missing relationship type is: HAS_ROOT)} {position: line: 9, column: 44, offset:
696} for query: '// Query already calculated and written node embeddings on nodes wit
h label in parameter $dependencies_projection_node including a communityId and centra
lity. Variables: dependencies_projection_node, dependencies_projection_write_propert
y. Requires "Add_file_name and_extension.cypher".\n \n  MATCH (codeUnit)\n  WHERE $de
pendencies_projection_node IN LABELS(codeUnit)\n    AND codeUnit[$dependencies_projec
tion_write_property] IS NOT NULL\n    // AND codeUnit.notExistingToForceRecalculation
IS NOT NULL // uncomment this line to force recalculation\n OPTIONAL MATCH (artifact:
Java:Artifact)-[:CONTAINS]->(codeUnit)\n    WITH *, artifact.name AS artifactName\n O
PTIONAL MATCH (projectRoot:Directory)<-[:HAS_ROOT]-(proj:TS:Project)-[:CONTAINS]->(co
deUnit)\n    WITH *, last(split(projectRoot.absoluteFileName, \'/\')) AS projectName
\n  RETURN DISTINCT \n         coalesce(codeUnit.fqn, codeUnit.globalFqn, codeUnit.fi
leName, codeUnit.signature, codeUnit.name) AS codeUnitName\n        ,codeUnit.name
AS shortCodeUnitName\n        ,coalesce(artifactName, projectName)
AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)           AS communi
tyId\n        ,coalesce(codeUnit.centralityPageRank, 0.01)       AS centrality\n
,codeUnit[$dependencies_projection_write_property] AS embedding\n   ORDER BY communit
yId'
```

```
The results have been provided by the query filename: ../cypher/Node_Embeddings/Node_
Embeddings_0a_Query_Calculated.cypher
```

	codeUnitName	shortCodeUnitName	projectName	communityId	centrality	embedding
0	org.axonframework.conversion	conversion	axon-conversion-5.0.3	0	1.501592	[0.3811345100402832, -0.1716684252023697, 0.59...
1	org.axonframework.conversion.jackson	jackson	axon-conversion-5.0.3	0	0.154765	[0.3668135404586792, -0.20763283967971802, 0.6...
2	org.axonframework.conversion.converter	converter	axon-conversion-5.0.3	0	0.150000	[0.2642297148704529, -0.14342153072357178, 0.5...
3	org.axonframework.conversion.jackson2	jackson2	axon-conversion-5.0.3	0	0.151059	[0.3026638627052307, -0.16931608319282532, 0.5...
4	org.axonframework.conversion.avro	avro	axon-conversion-5.0.3	0	0.152647	[0.310659795999527, -0.1719958484172821, 0.549...

1.3 Dimensionality reduction with Uniform Manifold Approximation and Projection (UMAP)

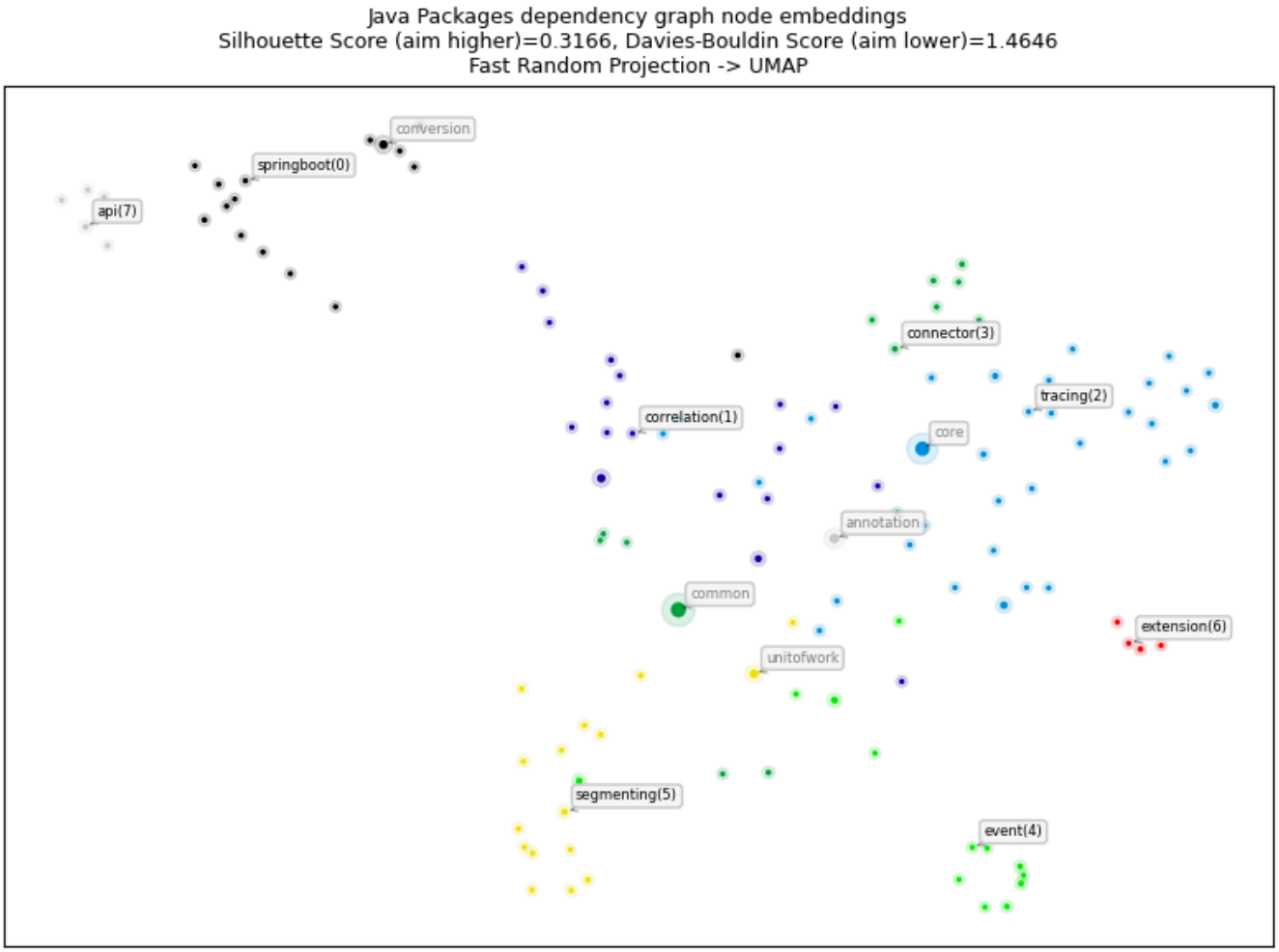
This step takes the original node embeddings in their high dimensionality, e.g. 32 floating point numbers, and reduces them into a two dimensional array for visualization. For more details look up the function "prepare_node_embeddings_for_2d_visualization".

About UMAP:

> The embedding is found by searching for a low dimensional projection of the data that has the closest possible equivalent fuzzy topological structure.

(see https://umap-learn.readthedocs.io)

1.4 Visualization of the node embeddings reduced to two dimensions



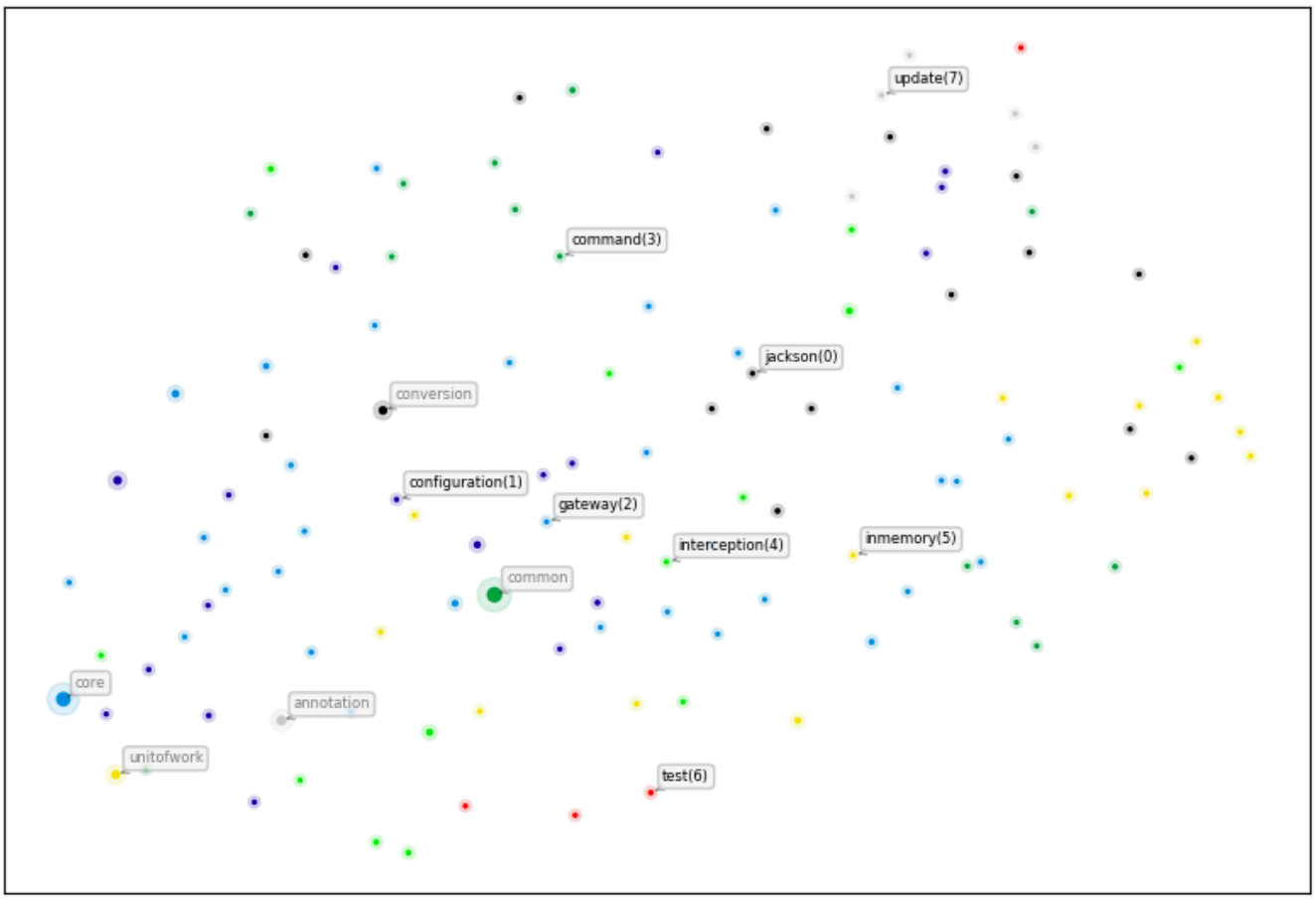
1.5 Node Embeddings for Java Packages using HashGNN

HashGNN resembles Graph Neural Networks (GNN) but does not include a model or require training. It combines ideas of GNNs and fast randomized algorithms. For more details see HashGNN. Here, the latter 3 steps are combined into one for HashGNN.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: HAS_ROOT)} {position: line: 9, column: 44, offset: 696} for query: '// Query already calculated and written node embeddings on nodes with label in parameter $dependencies_projection_node including a communityId and centrality. Variables: dependencies_projection_node, dependencies_projection_write_property. Requires "Add_file_name and_extension.cypher".\n \n  MATCH (codeUnit)\n  WHERE $dependencies_projection_node IN LABELS(codeUnit)\n    AND codeUnit[$dependencies_projection_write_property] IS NOT NULL\n    // AND codeUnit.notExistingToForceRecalculation IS NOT NULL // uncomment this line to force recalculation\n OPTIONAL MATCH (artifact:Java:Artifact)-[:CONTAINS]->(codeUnit)\n    WITH *, artifact.name AS artifactName\n OPTIONAL MATCH (projectRoot:Directory)<-[:HAS_ROOT]-(proj:TS:Project)-[:CONTAINS]->(codeUnit)\n    WITH *, last(split(projectRoot.absoluteFileName, \'/\')) AS projectName\n  RETURN DISTINCT \n         coalesce(codeUnit.fqn, codeUnit.globalFqn, codeUnit.fileName, codeUnit.signature, codeUnit.name) AS codeUnitName\n        ,codeUnit.name AS shortCodeUnitName\n        ,coalesce(artifactName, projectName) AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)           AS communityId\n        ,coalesce(codeUnit.centralityPageRank, 0.01)       AS centrality\n ,codeUnit[$dependencies_projection_write_property] AS embedding\n   ORDER BY communityId'
```

The results have been provided by the query filename: ../cypher/Node_Embeddings/Node_Embeddings_0a_Query_Calculated.cypher

	codeUnitName	shortCodeUnitName	projectName	communityId	centrality	embedding
0	org.axonframework.conversion	conversion	axon-conversion-5.0.3	0	1.501592	[-1.0825317353010178, 0.21650634706020355, -0....
1	org.axonframework.conversion.jackson	jackson	axon-conversion-5.0.3	0	0.154765	[-0.4330126941204071, 0.21650634706020355, -1....
2	org.axonframework.conversion.converter	converter	axon-conversion-5.0.3	0	0.150000	[0.21650634706020355, 0.0, -1.2990380823612213...
3	org.axonframework.conversion.jackson2	jackson2	axon-conversion-5.0.3	0	0.151059	[0.8660253882408142, 0.21650634706020355, -1.0...
4	org.axonframework.conversion.avro	avro	axon-conversion-5.0.3	0	0.152647	[0.8660253882408142, 0.21650634706020355, -1.0...

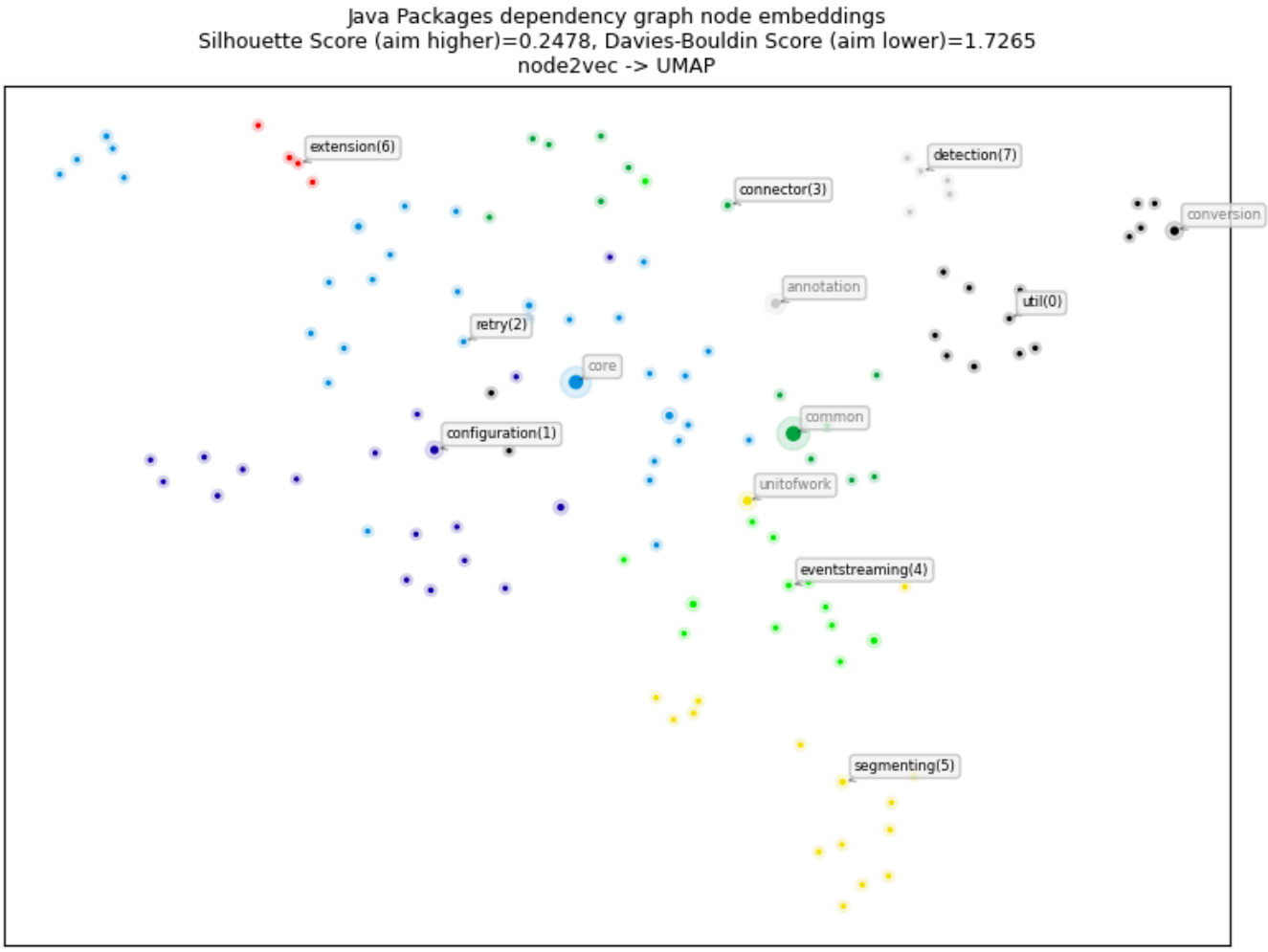


1.6 Node Embeddings for Java Packages using node2vec

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: HAS_ROOT)} {position: line: 9, column: 44, offset: 696} for query: '// Query already calculated and written node embeddings on nodes with label in parameter $dependencies_projection_node including a communityId and centrality. Variables: dependencies_projection_node, dependencies_projection_write_property. Requires "Add_file_name and_extension.cypher".\n \n  MATCH (codeUnit)\n  WHERE $dependencies_projection_node IN LABELS(codeUnit)\n    AND codeUnit[$dependencies_projection_write_property] IS NOT NULL\n    // AND codeUnit.notExistingToForceRecalculation IS NOT NULL // uncomment this line to force recalculation\n OPTIONAL MATCH (artifact:Java:Artifact)-[:CONTAINS]->(codeUnit)\n    WITH *, artifact.name AS artifactName\n OPTIONAL MATCH (projectRoot:Directory)<-[:HAS_ROOT]-(proj:TS:Project)-[:CONTAINS]->(codeUnit)\n    WITH *, last(split(projectRoot.absoluteFileName, \'/\')) AS projectName\n  RETURN DISTINCT \n         coalesce(codeUnit.fqn, codeUnit.globalFqn, codeUnit.fileName, codeUnit.signature, codeUnit.name) AS codeUnitName\n        ,codeUnit.name AS shortCodeUnitName\n        ,coalesce(artifactName, projectName) AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)           AS communityId\n        ,coalesce(codeUnit.centralityPageRank, 0.01)       AS centrality\n ,codeUnit[$dependencies_projection_write_property] AS embedding\n   ORDER BY communityId'
```

The results have been provided by the query filename: ../cypher/Node_Embeddings/Node_Embeddings_0a_Query_Calculated.cypher

	codeUnitName	shortCodeUnitName	projectName	communityId	centrality	embedding
0	org.axonframework.conversion	conversion	axon-conversion-5.0.3	0	1.501592	[0.5164653658866882, 0.49344944953918457, -0.5...
1	org.axonframework.conversion.jackson	jackson	axon-conversion-5.0.3	0	0.154765	[0.5367317795753479, 0.5524206757545471, -0.52...
2	org.axonframework.conversion.converter	converter	axon-conversion-5.0.3	0	0.150000	[0.7013564109802246, 0.6765453219413757, -0.50...
3	org.axonframework.conversion.jackson2	jackson2	axon-conversion-5.0.3	0	0.151059	[0.499202162027359, 0.5945873260498047, -0.463...
4	org.axonframework.conversion.avro	avro	axon-conversion-5.0.3	0	0.152647	[0.5667169094085693, 0.5885520577430725, -0.46...

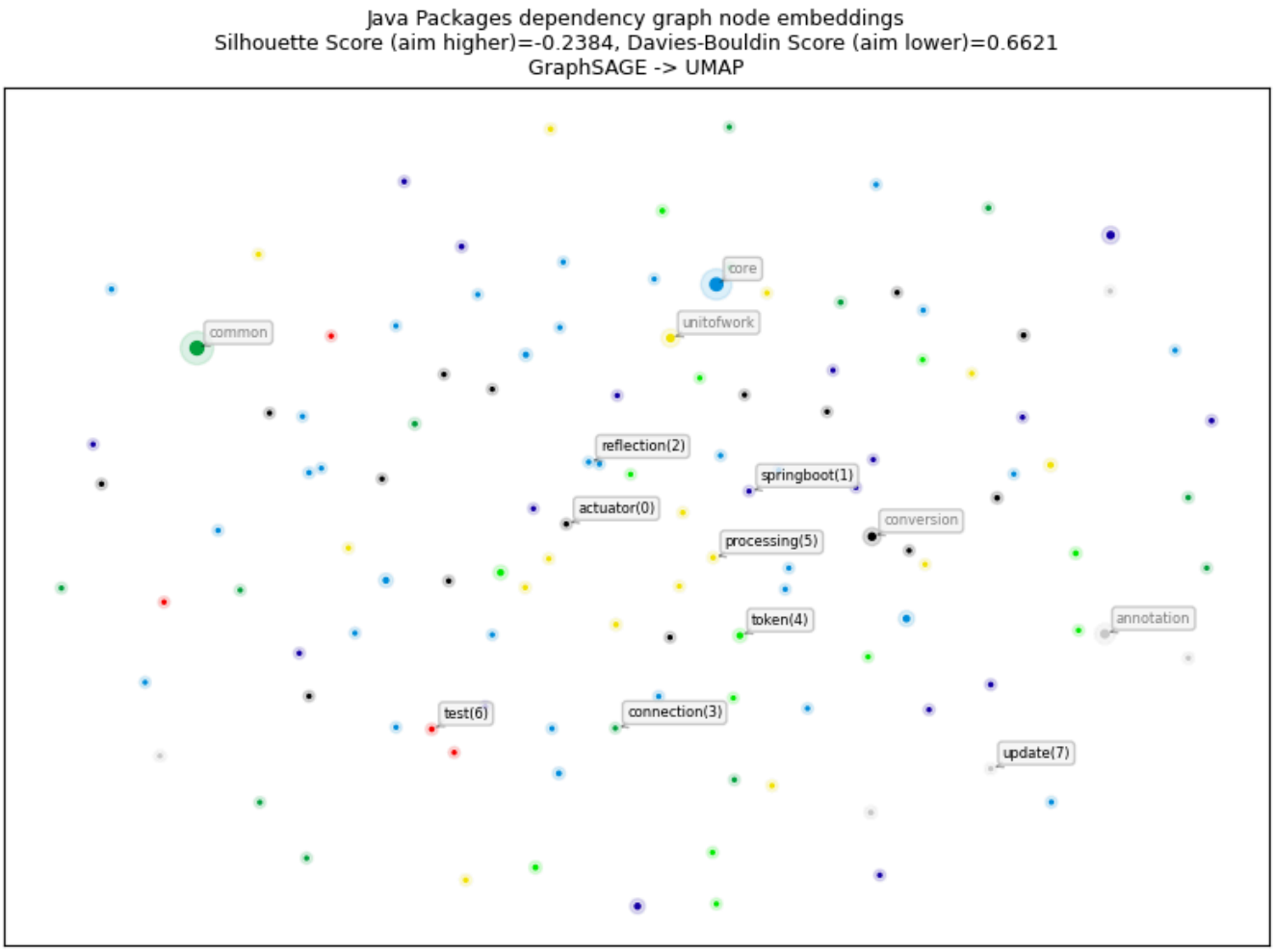


1.7 Node Embeddings for Java Packages using GraphSAGE

	modelName	didConverge	ranEpochs	epochLosses	trainingTimeMilliseconds
0	java-package-embeddings-notebook-graphSAGE	True	1	[27.239631432410533]	150

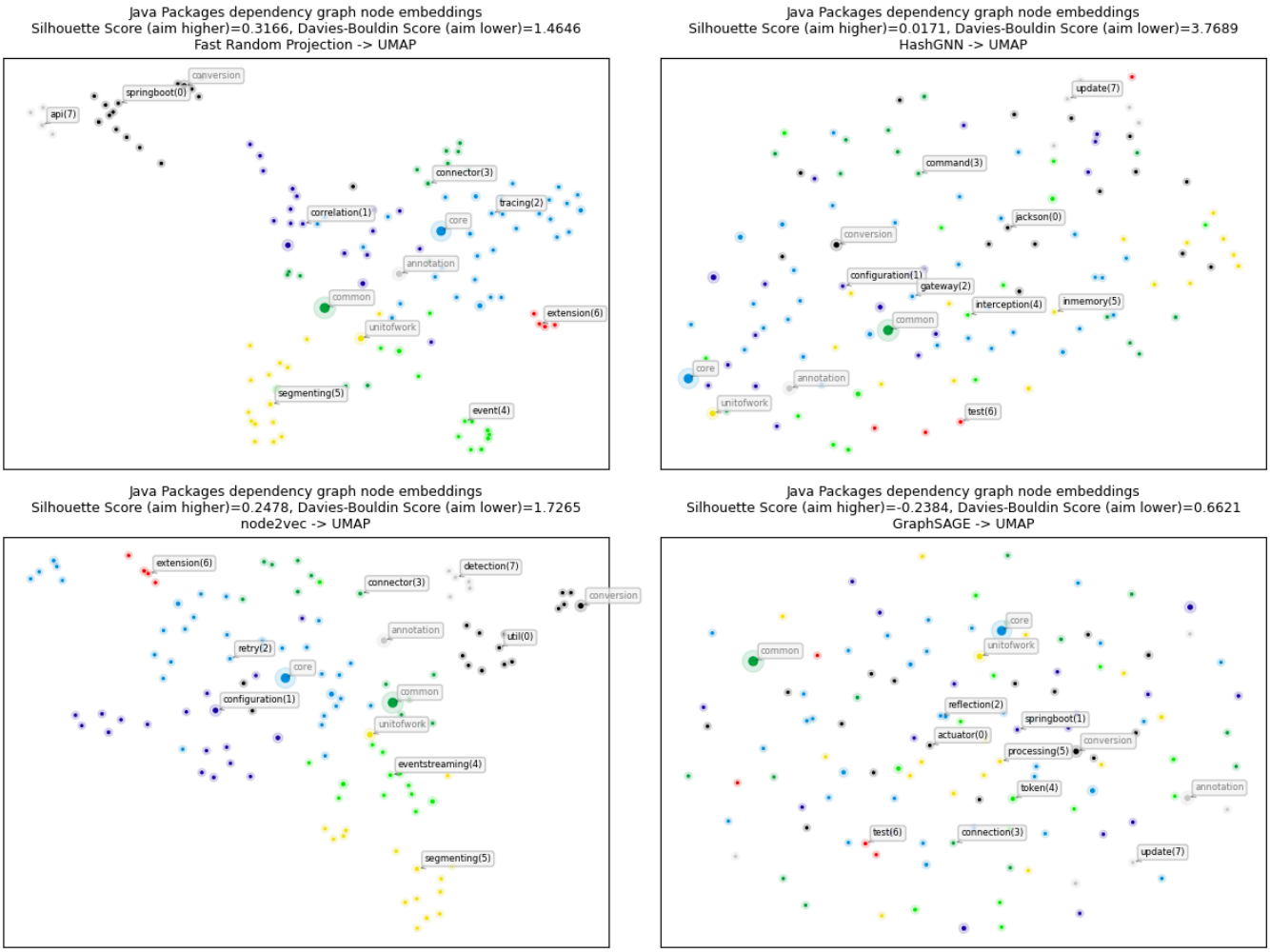
```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownRelationshipTypeWarning} {category: UNRECOGNIZED} {title: The provided relationship type is not in the database.} {description: One of the relationship types in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing relationship type is: HAS_ROOT)} {position: line: 13, column: 44, offset: 473} for query: '// Node Embeddings 4d using GraphSAGE: Stream. Requires "Add_file_name and_extension.cypher".\n \n CALL gds.beta.graphSage.stream(\n  $dependencies_projection + \'-cleaned\', {\n       modelName: $dependencies_projection + \'-graphSAGE\'\n   }\n )\n YIELD nodeId, embedding\n  WITH gds.util.asNode(nodeId) AS codeUnit\n ,embedding\n OPTIONAL MATCH (artifact:Java:Artifact)-[:CONTAINS]->(codeUnit)\n    WITH *, artifact.name AS artifactName\n OPTIONAL MATCH (projectRoot:Directory)<-[:HAS_ROOT]-(proj:TS:Project)-[:CONTAINS]->(codeUnit)\n    WITH *, last(split(projectRoot.absoluteFileName, \'/\')) AS projectName   \n  RETURN DISTINCT \n         coalesce(codeUnit.fqn, codeUnit.globalFqn, codeUnit.fileName, codeUnit.signature, codeUnit.name) AS codeUnitName\n        ,codeUnit.name                                  AS shortCodeUnitName\n        ,elementId(codeUnit)                            AS nodeElementId\n        ,coalesce(artifactName, projectName)         AS projectName\n        ,coalesce(codeUnit.communityLeidenId, 0)     AS communityId\n        ,coalesce(codeUnit.centralityPageRank, 0.01) AS centrality\n        ,embedding'
```

	codeUnitName	shortCodeUnitName	nodeElementId	projectName	communityId	centrality	embedding
0	org.axonframework.conversion	conversion	4:d44a0fae-8f59-46a6-9ba5-078f54dbbe8c:19	axon-conversion-5.0.3	0	1.501592	[-0.004854696704727069, 0.03805140358176101, 0...
1	org.axonframework.conversion.jackson	jackson	4:d44a0fae-8f59-46a6-9ba5-078f54dbbe8c:20	axon-conversion-5.0.3	0	0.154765	[-0.00485469670472707, 0.038051403581761026, 0...
2	org.axonframework.conversion.converter	converter	4:d44a0fae-8f59-46a6-9ba5-078f54dbbe8c:21	axon-conversion-5.0.3	0	0.150000	[-0.004854696704727068, 0.03805140358176098, 0...
3	org.axonframework.conversion.jackson2	jackson2	4:d44a0fae-8f59-46a6-9ba5-078f54dbbe8c:22	axon-conversion-5.0.3	0	0.151059	[-0.004854696704727069, 0.038051403581761, 0.1...
4	org.axonframework.conversion.avro	avro	4:d44a0fae-8f59-46a6-9ba5-078f54dbbe8c:23	axon-conversion-5.0.3	0	0.152647	[-0.00485469670472707, 0.03805140358176102, 0....



2. Compare Node Embeddings

In this section we will compare all node embedding methods from above in a grid plot. This helps to see how well the different algorithms were able to capture the structure of the graph and how well the communities are separated.



Interpreting Node Embedding Results

Summary of Observations

- **FastRP** and **node2vec** show clear, well-separated clusters
- **HashGNN** and **GraphSAGE** produce more diffuse embeddings
- Silhouette scores are high for FastRP / node2vec and low for HashGNN / GraphSAGE

These differences are expected and stem from the **fundamentally different objectives** of the algorithms.

Key Takeaways

- **FastRP and node2vec** are well-suited for **community discovery and visualization**
- **HashGNN** is best viewed as a **fast structural fingerprint**, not a clustering embedding
- **GraphSAGE** requires meaningful node features or labels and performs poorly in dense, feature-poor settings
- Poor silhouette scores for HashGNN and GraphSAGE are **expected and theoretically consistent**